Ben-Gurion University of the Negev
Faculty of Computer and Information Science

Deep Learning
Assignment 3

The purpose of the assignment

Enabling students to experiment with building a recurrent neural net and using it on a real-world dataset. In addition to practical knowledge in the "how to" of building the network, an additional goal is introducing the students to the challenge of integrating different sources of information into a single framework.

Submission instructions:

a) The assignment due date: 05.06.2026.
b) The assignment is to be carried out using PyTorch.
c) Submission in **pairs** only. Only **one copy** of the assignment needs to be uploaded to Moodle. The file name needs to be in the format "ID1_ID2.zip".
d) Plagiarism of any kind (e.g., GitHub) is forbidden.
e) The entire project will be submitted as a single zip file, containing both the report (in PDF form only) and the code. It is the students' responsibility to make sure that the file is valid (i.e. can be opened correctly).
f) The report accompanying the code needs to be a .**docx file** (use Calibri font, text size 12, margins of 2.5cm). The report itself will not exceed **6 pages** of text. Figures (graphs, tables, etc) can be placed an appendix, and must be referenced in the main body of report.

Introduction

In class we covered the topic of automatic sentence completion/generation. In addition to the completion of "regular" sentences, this technique can also be applied to other domains such as lyrics and melodies generation. In this task you will train a neural net to generate lyrics based on the provided melody.

During the training of the model you will have access both to the lyrics of a song and its melody. The melodies are stored in .mid (MIDI files) and contain various types of information – notes, the instruments

used etc. You are encouraged to experiment with various methods to incorporate this information with the lyrics. During the test phase, you are required to automatically generate lyrics for a provided melody.

Please note that this assignment cannot be measured using objective (i.e., absolute) performance measures. Instead, we will be evaluating your approach to the solution, the implementation, and your analysis of your model's performance.

Instructions

1. Please download the following:
    a. A .zip file containing all the MIDI files of the participating songs
    b. the .csv file with all the lyrics of the participating songs (600 train and 5 test)
    c. Pretty_Midi, a python library for the analysis of MIDI files
2. Implement a recurrent neural net (LSTM or GRU) to carry out the task described in the introduction.
    a. During each step of the training phase, your architecture will receive as input one word of the lyrics. Words are to be represented using the Word2Vec representation that can be found online (300 entries per term, as learned in class).
    b. The task of the network is to predict the next word of the song's lyrics. Please see Figure 1 for an illustration. You may use any loss function
    c. In addition to this textual information, you need to include information extracted from the MIDI file. The method for implementing this requirement is entirely up to your consideration. Figure 1 shows one of the more simplistic options – inserting the entire melody representation at each step.
    d. Note that your mechanism for selecting the next word should not be deterministic (i.e., always select the word with the highest probability) but rather be sampling-based. The likelihood of a term to be selected by the sampling should be proportional to its probability.
    e. Implement at least two additional decoding strategies on top of the basic proportional sampling described above: temperature-scaled sampling and either top-k or nucleus (top-p) sampling. These strategies should be selectable at generation time and must not require retraining the model.
    f. You may add whatever additions you want to the architecture (e.g., regularization, attention, teacher forcing)
    g. You may create a validation set. The manner of splitting (and all related decisions) are up to you.

h. You need to set “guidelines” (either fixed rules, preferably through the loss function) regarding the length of the generated text, the maximal number of words per line, etc. The goal is to make your lyrics look like those of an actual song.

3. The Pretty_Midi package offers multiple options for analyzing .mid files. Figures 2-4 demonstrate the types of information that can be gathered.
4. You can add whatever other information you consider relevant to further improve the performance of your model.

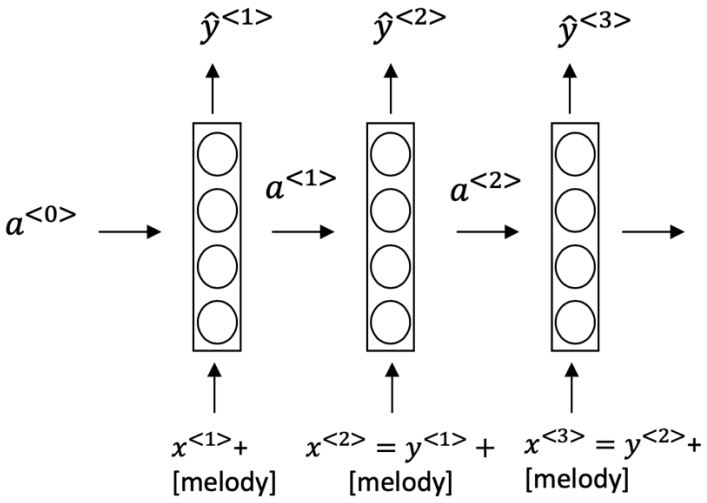

Figure 1: an example of a simplistic way of combining the melody and lyrics

5. You are to evaluate two approaches for integrating the melody information into your model. The two approaches don’t have to be completely different (one can build upon the other, for example), but please refrain from making only miniature changes.
6. In addition to the two melody-conditioned variants, train a lyrics-only baseline: an otherwise identical recurrent network that receives no melody information whatsoever. This baseline serves as a control, allowing you (and us) to assess what the melody integration actually contributes. Report its training and validation loss alongside those of the melody-conditioned variants.

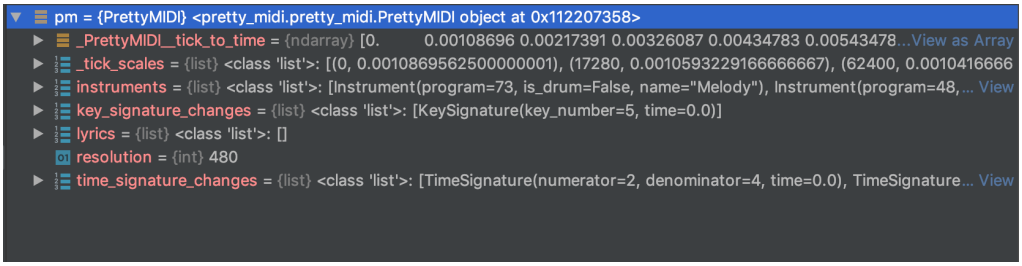

Figure 2: general .mid file information

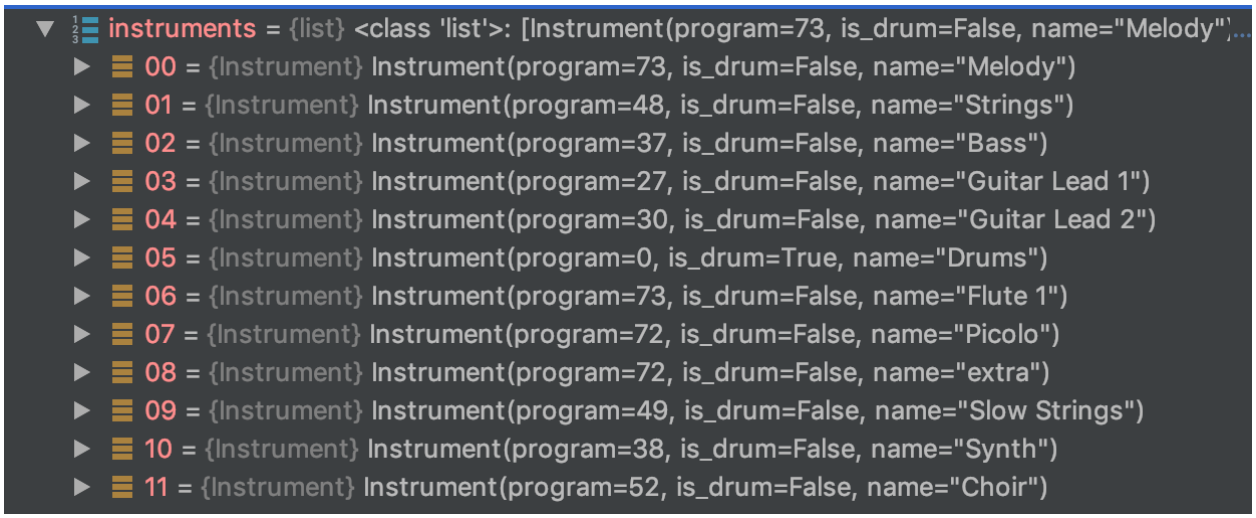

Figure 3: Instrument information of the analyzed file

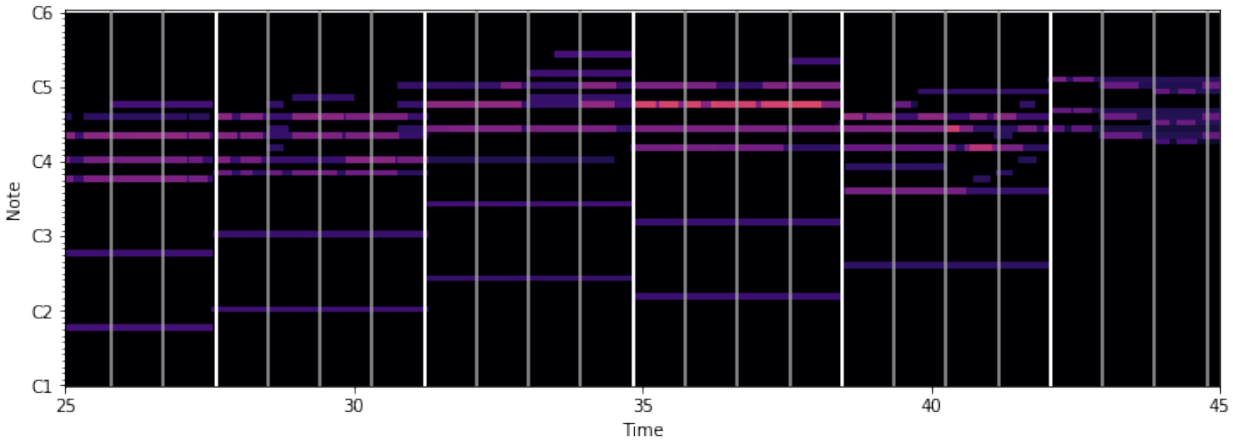

Figure 4: timing information of the analyzed file

7. Please include the following information in your report regarding the training phase:
    a. The chosen architecture of your model
    b. A clear description of your approach(s) for integrating the melody information together with the lyrics
    c. TensorBoard graphs showing the training and validation loss of all your models, including the lyrics-only baseline.
8. Please include the following information in your report regarding the test phase:
    a. For each of the melodies in the test set, produce the outputs (lyrics) for each of the two melody-conditioned variants you developed and for the lyrics-only baseline. The input should

be the melody (where applicable) and the initial word of the output lyrics. Include all generated lyrics in your submission.

b. For each melody, repeat the process described above three times, with different words (the same words should be used for all melodies).
c. Attempt to analyze the effect of the selection of the first word and/or melody on the generated lyrics.
d. Melody-influence probe: design and run a controlled experiment that tests whether the melody genuinely affects the output of your model, rather than being effectively ignored. For example, you may keep the initial word fixed while feeding the model a mismatched, shuffled, or otherwise corrupted melody, and measure how much the generated lyrics change relative to the output produced with the correct melody. You are required to define your own quantitative measure of how much the output changed, and to justify your choice. Report the results for both melody-conditioned variants and discuss what they imply about the effectiveness of your melody integration.
e. Compare the decoding strategies you implemented. For a small fixed subset of the test cases (e.g., two melodies), generate lyrics using proportional, temperature-scaled, and top-k or nucleus sampling, and analyze the resulting trade-off between diversity and coherence using concrete examples from your own outputs. State which strategy you would choose for this task, and why.
f. Inspect the lyrics generated by your models and identify at least three distinct failure modes (for example: degenerate repetition, loss of grammatical coherence, collapse of the line structure, or drifting off-topic). For each failure mode, provide a short concrete example taken from your own generated output, and briefly hypothesize its cause.

Good Luck!